

- Description

This module manages a smart road ice detection and anti-icing system, ensuring that:

1. **The road surface temperature is continuously monitored to detect ice formation risks.**
2. **If the temperature drops below freezing (0°C), anti-icing salt is automatically dispersed.**
3. **If the temperature rises above a safe threshold, salt dispersion is stopped to prevent waste.**
4. **A log of all temperature readings, salt dispersion events, and system activations is maintained for road safety optimization.**

This system helps reduce road accidents, improve driving safety, and optimize salt usage for cost efficiency.

- Specification

module SmartRoadIceDetectionEx

var currentTemperature, freezingPoint, stopSaltDispersionTemp: real

var saltDispersionActive: boolean

process updateRoadTemperature(changeAmount: real)

functional scenario form:

currentTemperature >= freezingPoint && Road temperature maintained.

||

currentTemperature < freezingPoint && saltDispersionActive = true, Salt dispersion activated.

||

currentTemperature > stopSaltDispersionTemp && saltDispersionActive = false
Salt dispersion deactivated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.*;
```

```

import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c127_SmartRoadSafetySystemEx
 * Domain: Intelligent anti-icing salt dispersion for road safety
 *
 * Features:
 * - Core SmartRoadSafety logic (temperature-
based salt dispersion with hysteresis)
 * - Sensors: road temperature, humidity, precipitation, friction
 * - Controller: SaltDispenserController (ON/OFF with thresholds)
 * - KPIs: salt usage estimation, cost
 * - Alert system & file-backed logger (append mode)
 * - Swing GUI: real-
time display, controls, simulation panel, log tail
 * - Simulation engine (weather: ambient temp, wind chill, humidity, p
recipitation)
 * - History replay (log playback)
 * - CLI harness (VT/FT/Z3/UVT) with comment-to-toggle style
 *
 * Compile: javac code/c127_SmartRoadSafetySystemEx.java
 * Run      : java code.c127_SmartRoadSafetySystemEx
 * CLI      : java code.c127_SmartRoadSafetySystemEx --cli
 * =====
 */
/* ===== Utilities ===== */
class RStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").for
mat(v); }
    static String fmt3(double v) { return new DecimalFormat("0.000").fo
rmat(v); }
    static String now() { return LocalDateTime.now().format(DateTimeFor
matter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
}
/* ===== Logger ===== */
class RDataLogger {
    private final File file;
    private final ConcurrentLinkedQueue<String> inMemory = new Concurr
entLinkedQueue<>();
    private final int inMemoryCap;
    RDataLogger(String path, int inMemoryCap) {
        this.file = new File(path);
        this.inMemoryCap = Math.max(50, inMemoryCap);
    }
}

```

```

        try {
            if (!file.exists()) {
                File parent = file.getParentFile();
                if (parent != null) parent.mkdirs();
                file.createNewFile();
            }
        } catch (IOException e) {
            System.err.println("[Logger] Failed to create log file: " +
e.getMessage());
        }
    }

    synchronized void log(String message) {
        String line = "[" + RStr.now() + "]" + message;
        inMemory.add(line);
        while (inMemory.size() > inMemoryCap) inMemory.poll();
        try (FileOutputStream fos = new FileOutputStream(file, true);
            OutputStreamWriter osw = new OutputStreamWriter(fos, Stand
ardCharsets.UTF_8);
            BufferedWriter bw = new BufferedWriter(osw)) {
            bw.write(line);
            bw.newLine();
        } catch (IOException e) {
            System.err.println("[Logger] Write failed: " + e.getMessage
());
        }
    }

    List<String> recent(int max) {
        ArrayList<String> list = new ArrayList<>(inMemory);
        if (list.size() <= max) return list;
        return list.subList(list.size() - max, list.size());
    }

    File getFile() { return file; }
}

/* ===== Alerts ===== */
class RAlertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
        Alert(Severity s, String m) { this.severity = s; this.message =
m; }
        @Override public String toString() {
            return "[" + time + "]" + severity + "]" + message;

```

```

    }
}
private final List<Alert> alerts = new ArrayList<>();
private final RDataLogger logger;
RAlertSystem(RDataLogger logger) { this.logger = logger; }
void push(Severity s, String msg) {
    Alert a = new Alert(s, msg);
    alerts.add(a);
    logger.log("ALERT " + s + " :: " + msg);
    System.out.println(a);
}
List<Alert> recent(int n) {
    if (alerts.size() <= n) return new ArrayList<>(alerts);
    return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
}
}
/* ===== Sensors ===== */
abstract class RSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    RSensor(String name) { this.name = name; }
    abstract void update(RoadContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}
/** Shared context between core and sensors. */
class RoadContext {
    public double currentTemperature; // °C
    public double freezingPoint; // °C
    public double stopSaltDispersionTemp; // °C
    public boolean saltDispersionActive;
    public double humidity; // 0~100%
    public double precipitationRate; // mm/h
    public boolean snowing; // true if snow precipitation
    public double friction; // 0~1 (1=high friction dry, 0=sl
ippery)
    public double ambientTemperature; // ambient air °C
    public double windSpeed; // m/s

```

```

RoadContext cloneCopy() {
    RoadContext c = new RoadContext();
    c.currentTemperature = currentTemperature;
    c.freezingPoint = freezingPoint;
    c.stopSaltDispersionTemp = stopSaltDispersionTemp;
    c.saltDispersionActive = saltDispersionActive;
    c.humidity = humidity;
    c.precipitationRate = precipitationRate;
    c.snowing = snowing;
    c.friction = friction;
    c.ambientTemperature = ambientTemperature;
    c.windSpeed = windSpeed;
    return c;
}
}

class RoadTemperatureSensor extends RSensor {
    private double drift = 0.0; // slow bias drift
    RoadTemperatureSensor() { super("RoadTemperatureSensor"); }
    @Override
    void update(RoadContext ctx) {
        drift += (rnd.nextDouble() - 0.5) * 0.01;
        value = ctx.currentTemperature + drift + (rnd.nextDouble() - 0.
5) * 0.1;
        ok = (value > -60 && value < 60);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class HumiditySensor extends RSensor {
    HumiditySensor() { super("HumiditySensor"); }
    @Override
    void update(RoadContext ctx) {
        value = Math.max(0, Math.min(100, ctx.humidity + (rnd.nextDoubl
e() - 0.5) * 1.5));
        ok = (value >= 0 && value <= 100);
        status = ok ? "OK" : "BROKEN";
    }
}

class PrecipitationSensor extends RSensor {
    private boolean snowing;
    PrecipitationSensor() { super("PrecipitationSensor"); }
    @Override
    void update(RoadContext ctx) {
        double noise = (rnd.nextDouble() - 0.5) * 0.2;
        value = Math.max(0, ctx.precipitationRate + noise); // mm/h
    }
}

```

```

        snowing = ctx.snowing;
        ok = true;
        status = snowing ? "SNOW" : (value > 0 ? "RAIN" : "NONE");
    }
    boolean isSnowing() { return snowing; }
}

class FrictionSensor extends RSensor {
    FrictionSensor() { super("FrictionSensor"); }
    @Override
    void update(RoadContext ctx) {
        // lower friction when icy, wet, or snowing
        double base = 0.9;
        base -=
= Math.max(0, (0 - Math.min(0, ctx.currentTemperature - ctx.freezingPoi
nt))) * 0.03; // below freezing -> lower
        base -= Math.min(0.4, ctx.precipitationRate * 0.02);
        if (ctx.snowing) base -= 0.2;
        base = Math.max(0.1, Math.min(1.0, base + (rnd.nextDouble() - 0
.5) * 0.03));
        value = base;
        ok = (value >= 0.0 && value <= 1.0);
        status = ok ? (value >= 0.6 ? "GOOD" : (value >= 0.4 ? "CAUTION
" : "DANGEROUS")) : "BROKEN";
    }
}

/* ===== Controller ===== */
class SaltDispenserController {
    private boolean active = false;
    private double onThreshold; // below -> ON
    private double offThreshold; // above -> OFF
    private double gramsPerSecond = 200.0; // dispenser rate
    private double totalSaltGrams = 0.0;
    private double saltCostPerKg = 0.08; // hypothetical cost
    SaltDispenserController(double freezeOn, double stopOff) {
        this.onThreshold = Math.min(freezeOn, stopOff - 0.1);
        this.offThreshold = Math.max(stopOff, freezeOn + 0.1);
    }
    void update(double currentTemperature) {
        if (currentTemperature < onThreshold) active = true;
        else if (currentTemperature > offThreshold) active = false;
        // between: keep previous state
    }
    void addTimeStep(double seconds) {
        if (active) totalSaltGrams += gramsPerSecond * seconds;
    }
}

```

```

    }
    void setThresholds(double freezeOn, double stopOff) {
        this.onThreshold = Math.min(freezeOn, stopOff - 0.1);
        this.offThreshold = Math.max(stopOff, freezeOn + 0.1);
    }
    boolean isActive() { return active; }
    void forceOn() { active = true; }
    void forceOff() { active = false; }
    double getTotalSaltKg() { return totalSaltGrams / 1000.0; }
    double getTotalSaltGrams() { return totalSaltGrams; }
    void resetSaltUsage() { totalSaltGrams = 0.0; }
    double getOnThreshold() { return onThreshold; }
    double getOffThreshold() { return offThreshold; }
    double getCost() { return (totalSaltGrams / 1000.0) * saltCostPerKg; }
; }
}
/* ===== Core Domain ===== */
class SmartRoadSafety {
    private final RDataLogger logger;
    private final RAlertSystem alert;
    // thresholds
    private double freezingPoint;           // °C: below => start dispersion
    private double stopSaltDispersionTemp; // °C: above => stop dispersion
    private double currentTemperature;      // °C
    // environment
    private double ambientTemperature;      // °C
    private double humidity;                // %
    private double precipitationRate;       // mm/h
    private boolean snowing;                // true if snow
    private double friction;                // 0~1
    private double windSpeed;               // m/s
    // components
    private final SaltDispenserController dispenser;
    private final RoadTemperatureSensor tempSensor = new RoadTemperatureSensor();
    private final HumiditySensor humiditySensor = new HumiditySensor();
    private final PrecipitationSensor precipSensor = new PrecipitationSensor();
    private final FrictionSensor frictionSensor = new FrictionSensor();
    // listeners/UI
    private final List<Runnable> listeners = new ArrayList<>();

```

```

    SmartRoadSafety(double freezingPoint, double stopSaltDispersionTemp
, double initialTemperature,
                    RDataLogger logger, RAlertSystem alert) {
        this.freezingPoint = freezingPoint;
        this.stopSaltDispersionTemp = stopSaltDispersionTemp;
        this.currentTemperature = initialTemperature;
        this.logger = logger;
        this.alert = alert;
        // default environment
        this.ambientTemperature = initialTemperature + 1.5;
        this.humidity = 65;
        this.precipitationRate = 0;
        this.snowing = false;
        this.friction = 0.9;
        this.windSpeed = 1.2;
        this.dispenser = new SaltDispenserController(freezingPoint, sto
pSaltDispersionTemp);
        logger.log("SmartRoadSafety initialized. Temp=" + RStr.fmt2(cur
rentTemperature)
                    + "°C (freeze=" + RStr.fmt2(freezingPoint) + ", stop=" +
RStr.fmt2(stopSaltDispersionTemp) + ")");
    }
    void addListener(Runnable r) { listeners.add(r); }
    private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
    /* ===== Public API (original updateRoadTemperature) ===== */
    public synchronized void updateRoadTemperature(double changeAmount)
    {
        currentTemperature += changeAmount;
        manageSaltDispersion();
        sampleSensors();
        logger.log("Current Road Temperature: " + RStr.fmt2(currentTemp
erature) + "°C | Dispenser: " +
                    (dispenser.isActive() ? "ON" : "OFF"));
        System.out.printf("Current Road Temperature: %.2f°C\n", current
Temperature);
        System.out.println(dispenser.isActive() ? "Anti-
Icing Salt Dispersion Activated."
                                : "Anti-
Icing Salt Dispersion Deactivated.");
        emit();
    }
    /* ===== Control ===== */
    private void manageSaltDispersion() {

```

```

        dispenser.update(currentTemperature);
        if (currentTemperature < freezingPoint) {
            alert.push(RAlertSystem.Severity.WARNING,
                "Below freezing point: " + RStr.fmt2(currentTemperature) + "°C < " + RStr.fmt2(freezingPoint) + "°C");
        }
        if (currentTemperature > stopSaltDispersionTemp) {
            alert.push(RAlertSystem.Severity.INFO,
                "Above stop temperature: " + RStr.fmt2(currentTemperature) + "°C > " + RStr.fmt2(stopSaltDispersionTemp) + "°C");
        }
    }
    /* ===== Sensors ===== */
    private void sampleSensors() {
        RoadContext ctx = buildContext();
        tempSensor.update(ctx);
        humiditySensor.update(ctx);
        precipSensor.update(ctx);
        frictionSensor.update(ctx);
        // alert on dangerous friction
        if (frictionSensor.getValue() < 0.4) {
            alert.push(RAlertSystem.Severity.CRITICAL,
                "Dangerous low friction: " + RStr.fmt2(frictionSensor.getValue()));
        }
    }
    RoadContext buildContext() {
        RoadContext c = new RoadContext();
        c.currentTemperature = currentTemperature;
        c.freezingPoint = freezingPoint;
        c.stopSaltDispersionTemp = stopSaltDispersionTemp;
        c.saltDispersionActive = dispenser.isActive();
        c.humidity = humidity;
        c.precipitationRate = precipitationRate;
        c.snowing = snowing;
        c.friction = friction;
        c.ambientTemperature = ambientTemperature;
        c.windSpeed = windSpeed;
        return c;
    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // salt usage accumulation
        dispenser.addTimeStep(seconds);
    }

```

```

        // friction re-evaluation each tick (coarse)
        friction = Math.max(0.1, Math.min(1.0, friction + (dispenser.is
Active() ? +0.01 : -0.005)));
        sampleSensors();
        emit();
    }
    synchronized void applyWeather(double tempDelta, double humidityDel
ta,
                                double precipDelta, boolean snowFlag
, double windDelta) {
        currentTemperature += tempDelta;
        ambientTemperature += tempDelta * 0.6;
        humidity = clamp(humidity + humidityDelta, 0, 100);
        precipitationRate = Math.max(0, precipitationRate + precipDelta
);

        snowing = snowFlag;
        windSpeed = Math.max(0, windSpeed + windDelta);
        // wet/slippery logic
        if (precipitationRate > 0 || snowing) {
            friction -= 0.02 + Math.min(0.3, precipitationRate * 0.01);
        } else {
            friction += 0.01;
        }
        friction = clamp(friction, 0.1, 1.0);
        manageSaltDispersion();
        sampleSensors();
        logger.log(String.format(Locale.US,
            "Weather step: dT=%.3f, dH=%.3f, dP=%.3f, snow=%s, dWin
d=%.3f | roadT=%.2f, ambT=%.2f, hum=%.1f%%, P=%.2f, fr=%.2f",
            tempDelta, humidityDelta, precipDelta, String.valueOf(s
nowFlag), windDelta,
            currentTemperature, ambientTemperature, humidity, preci
pitationRate, friction));
        emit();
    }
    /* ===== Maintenance ===== */
    synchronized void calibrateTemp(double observed) {
        double old = currentTemperature;
        currentTemperature = observed;
        logger.log("Calibration: roadT " + RStr.fmt2(old) + " -> " + RS
tr.fmt2(currentTemperature));
        sampleSensors();
        emit();
    }
}

```

```

        synchronized void setThresholds(double freezePoint, double stopTemp
    ) {
        double oldF = freezingPoint, oldS = stopSaltDispersionTemp;
        freezingPoint = Math.min(freezePoint, stopTemp - 0.1);
        stopSaltDispersionTemp = Math.max(stopTemp, freezePoint + 0.1);
        dispenser.setThresholds(freezingPoint, stopSaltDispersionTemp);
        logger.log("Thresholds changed: freeze " + RStr.fmt2(oldF) + "-
>" + RStr.fmt2(freezingPoint) +
            ", stop " + RStr.fmt2(oldS) + "->" + RStr.fmt2(stopSalt
DispersionTemp));
        sampleSensors();
        emit();
    }
    /* ===== Getters for GUI ===== */
    double getCurrentTemperature() { return currentTemperature; }
    double getFreezingPoint() { return freezingPoint; }
    double getStopTemp() { return stopSaltDispersionTemp; }
    boolean isDispenserActive() { return dispenser.isActive(); }
    double getSaltKg() { return dispenser.getTotalSaltKg(); }
    double getSaltCost() { return dispenser.getCost(); }
    double getAmbientTemperature() { return ambientTemperature; }
    double getHumidity() { return humidity; }
    double getPrecipitationRate() { return precipitationRate; }
    boolean isSnowing() { return snowing; }
    double getFriction() { return friction; }
    double getWindSpeed() { return windSpeed; }
    RoadTemperatureSensor tempSensor() { return tempSensor; }
    HumiditySensor humiditySensor() { return humiditySensor; }
    PrecipitationSensor precipSensor() { return precipSensor; }
    FrictionSensor frictionSensor() { return frictionSensor; }
    private static double clamp(double v, double lo, double hi) { retur
n Math.max(lo, Math.min(hi, v)); }
}
/* ===== Simulation Engine =====
= */
class WeatherSimulationEngine implements Runnable {
    private final SmartRoadSafety road;
    private final RDataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    private volatile double tempMeanStep = -
0.02;    // road temp drift per tick
    private volatile double humidityMeanStep = +0.1;    // %
    private volatile double precipMeanStep = +0.02;    // mm/h

```

```

private volatile double windMeanStep = +0.02;    // m/s
private volatile double tickSeconds = 1.0;       // seconds per ti
ck
private volatile long sleepMillis = 250L;        // wall clock del
ay
private volatile boolean randomize = true;
private final Random rnd = new Random();
WeatherSimulationEngine(SmartRoadSafety road, RDataLogger logger) {
    this.road = road;
    this.logger = logger;
}
void start() {
    if (running.get()) return;
    running.set(true);
    thread = new Thread(this, "WeatherEngine");
    thread.setDaemon(true);
    thread.start();
    logger.log("Weather simulation started.");
}
void stop() {
    running.set(false);
    if (thread != null) {
        try { thread.join(2000); } catch (InterruptedException igno
red) {}
    }
    logger.log("Weather simulation stopped.");
}
boolean isRunning() { return running.get(); }
void setParams(double tempStep, double humStep, double precipStep,
double windStep,
                double tickSeconds, long delayMs, boolean randomize)
{
    this.tempMeanStep = tempStep;
    this.humidityMeanStep = humStep;
    this.precipMeanStep = precipStep;
    this.windMeanStep = windStep;
    this.tickSeconds = Math.max(0.1, tickSeconds);
    this.sleepMillis = Math.max(20, delayMs);
    this.randomize = randomize;
    logger.log(String.format(Locale.US,
        "Sim params updated: dT=%.4f, dH=%.4f, dP=%.4f, dWind=%
.4f, dt=%.2fs, delay=%dms, random=%s",
        tempStep, humStep, precipStep, windStep, this.tickSecon
ds, this.sleepMillis, String.valueOf(this.randomize)));
}

```

```

    }
    @Override
    public void run() {
        while (running.get()) {
            double dT = tempMeanStep;
            double dH = humidityMeanStep;
            double dP = precipMeanStep;
            double dW = windMeanStep;
            boolean snow = false;
            if (randomize) {
                dT += (rnd.nextDouble() - 0.5) * Math.abs(tempMeanStep)
* 1.0;
                dH += (rnd.nextDouble() - 0.5) * Math.abs(humidityMeanS
tep) * 2.0;
                dP = Math.max(0, dP + (rnd.nextDouble() - 0.5) * Math.m
ax(0.1, precipMeanStep * 4));
                dW = Math.max(0, dW + (rnd.nextDouble() - 0.5) * Math.m
ax(0.1, windMeanStep * 4));
                snow = rnd.nextDouble() < 0.15; // sometimes snow
            }
            road.applyWeather(dT, dH, dP, snow, dW);
            road.tickSeconds(tickSeconds);
            try {
                Thread.sleep(sleepMillis);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}

/* ===== Replay (History) =====
*/

class RoadReplayEngine {
    private final SmartRoadSafety road;
    private final RDataLogger logger;
    RoadReplayEngine(SmartRoadSafety road, RDataLogger logger) {
        this.road = road;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
    }
}

```

```

        logger.log("Replay started from: " + logFile.getAbsolutePath())
;
        try (BufferedReader br = new BufferedReader(new InputStreamReader(
new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Weather step:")) continue;
                try {
                    int idx = line.indexOf("Weather step:");
                    String seg = line.substring(idx);
                    // naive parse by tokens
                    boolean snow = seg.contains("snow=true");
                    String[] tokens = seg.split("\\|")[0].split(",");
                    double dT = Double.parseDouble(tokens[0].split("=")
[1].trim());
                    double dH = Double.parseDouble(tokens[1].split("=")
[1].trim());
                    double dP = Double.parseDouble(tokens[2].split("=")
[1].trim());
                    double dW = Double.parseDouble(tokens[4].split("=")
[1].trim());

                    road.applyWeather(dT, dH, dP, snow, dW);
                    road.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) { }
            }
        } catch (IOException e) {
            System.err.println("[Replay] Error: " + e.getMessage());
        }
        logger.log("Replay finished.");
    }
}

/* ===== GUI ===== */
class RoadGUI extends JFrame {
    private final SmartRoadSafety road;
    private final RDataLogger logger;
    private final RAlertSystem alerts;
    private final WeatherSimulationEngine engine;
    private final RoadReplayEngine replay;
    private JLabel lblRoadT, lblFreeze, lblStop, lblDisp, lblAmbT, lblHum,
    lblPrec, lblSnow, lblFr, lblWind, lblSalt, lblCost;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog;

```

```

private JSpinner spFreeze, spStop;
private JSpinner spDT, spDH, spDP, spDW, spSimDT, spDelay;
private JCheckBox cbRandomize;
private JButton btnStart, btnStop, btnReplay, btnCalib, btnAddT, btnSubT, btnResetSalt;
private javax.swing.Timer uiTimer;
RoadGUI(SmartRoadSafety road, RDataLogger logger, RAlertSystem alerts,
        WeatherSimulationEngine engine, RoadReplayEngine replay) {
    super("SmartRoadSafety - Anti-icing Salt Control");
    this.road = road;
    this.logger = logger;
    this.alerts = alerts;
    this.engine = engine;
    this.replay = replay;
    setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    buildUi();
    setSize(1180, 760);
    setLocationRelativeTo(null);
    uiTimer = new javax.swing.Timer(500, e -> refreshUi());
    uiTimer.start();
    road.addListener(this::refreshUi);
}

private JPanel titled(String title, JComponent inner) {
    JPanel p = new JPanel(new BorderLayout());
    p.setBorder(BorderFactory.createTitledBorder(title));
    p.add(inner, BorderLayout.CENTER);
    return p;
}

private void buildUi() {
    JPanel root = new JPanel(new BorderLayout(10,10));
    root.setBorder(new EmptyBorder(10,10,10,10));
    setContentPane(root);
    /* LEFT: Status & Controls */
    JPanel left = new JPanel();
    left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
    JPanel status = new JPanel(new GridLayout(0,2,6,6));
    lblRoadT = new JLabel("-");
    lblFreeze = new JLabel("-");
    lblStop = new JLabel("-");
    lblDisp = new JLabel("-");
    lblAmbT = new JLabel("-");
    lblHum = new JLabel("-");
    lblPrec = new JLabel("-");

```

```

        lblSnow = new JLabel("-");
        lblFr = new JLabel("-");
        lblWind = new JLabel("-");
        lblSalt = new JLabel("-");
        lblCost = new JLabel("-");
        status.add(new JLabel("Road Temp (°C):")); status.add(lblRoadT)
;
        status.add(new JLabel("Freeze (°C):")); status.add(lblFreeze)
;

        status.add(new JLabel("Stop (°C):")); status.add(lblStop);
        status.add(new JLabel("Dispenser:")); status.add(lblDisp);
        status.add(new JLabel("Ambient (°C):")); status.add(lblAmbT);
        status.add(new JLabel("Humidity (%):")); status.add(lblHum);
        status.add(new JLabel("Precip (mm/h):")); status.add(lblPrec);
        status.add(new JLabel("Snowing:")); status.add(lblSnow);
        status.add(new JLabel("Friction:")); status.add(lblFr);
        status.add(new JLabel("Wind (m/s):")); status.add(lblWind);
        status.add(new JLabel("Salt (kg):")); status.add(lblSalt);
        status.add(new JLabel("Cost ($)")); status.add(lblCost);
        left.add(titled("Live Status", status));
        JPanel th = new JPanel(new GridLayout(0,2,6,6));
        spFreeze = new JSpinner(new SpinnerNumberModel(0.0, -
50.0, 20.0, 0.1));
        spStop = new JSpinner(new SpinnerNumberModel(5.0, -
50.0, 30.0, 0.1));
        JButton btnApply = new JButton("Apply Thresholds");
        btnApply.addActionListener(e -> {
            double f = ((Number)spFreeze.getValue()).doubleValue();
            double s = ((Number)spStop.getValue()).doubleValue();
            road.setThresholds(f, s);
        });

//      JPanel th = new JPanel(new GridLayout(0,2,6,6));
//      spFreeze = new JSpinner(new SpinnerNumberModel(0.0, -
//      50.0, 20.0, 0.1));
//      spStop = new JSpinner(new SpinnerNumberModel(5.0, -
//      50.0, 30.0, 0.1));
//      JButton btnApply = new JButton("Apply Thresholds");
//      btnApply.addActionListener(e -> {
//          double f = ((Number)spFreeze.getValue()).doubleValue();
//          double s = ((Number)spStop.getValue()).doubleValue();
//          road.setThresholds(f, s);
//      });

```

```

        th.add(new JLabel("Freezing point (°C)")); th.add(spFreeze);
        th.add(new JLabel("Stop temp (°C)"));      th.add(spStop);
        th.add(new JLabel(" "));                    th.add(btnApply);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Thresholds", th));
        JPanel ops = new JPanel(new GridLayout(0,2,6,6));
        btnAddT = new JButton("Raise Temp +1.0°C");
        btnSubT = new JButton("Lower Temp -1.0°C");
        btnCalib = new JButton("Calibrate Temp");
        btnResetSalt = new JButton("Reset Salt Usage");
        btnAddT.addActionListener(e -> road.updateRoadTemperature(+1.0)
);
        btnSubT.addActionListener(e -> road.updateRoadTemperature(-
1.0));
        btnCalib.addActionListener(e -> {
            String s = JOptionPane.showInputDialog(this, "Observed road
temp (°C):", RStr.fmt2(road.getCurrentTemperature()));
            if (s == null) return;
            try { road.calibrateTemp(Double.parseDouble(s)); }
            catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
        });
        btnResetSalt.addActionListener(e -> {
            // small helper through thresholds reapply: no public reset
            // on controller, so push via temp tick
            // alternatively we could add a method in SmartRoadSafety t
            // o reset, but we keep it simple:
            try {
                java.lang.reflect.Field fCtrl = SmartRoadSafety.class.g
etDeclaredField("dispenser");
                fCtrl.setAccessible(true);
                SaltDispenserController ctrl = (SaltDispenserController
) fCtrl.get(road);
                ctrl.resetSaltUsage();
                logger.log("Salt usage reset by user.");
            } catch (Throwable ex) {
                JOptionPane.showMessageDialog(this, "Reset failed: " +
ex.getMessage());
            }
        });
        ops.add(btnAddT); ops.add(btnSubT);
        ops.add(btnCalib); ops.add(btnResetSalt);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Operations", ops));

```

```

//      JPanel sim = new JPanel(new GridLayout(0,2,6,6));
//      spDT = new JSpinner(new SpinnerNumberModel(-0.02, -
5.0, 5.0, 0.01));
//      spDH = new JSpinner(new SpinnerNumberModel(+0.10, -
5.0, 5.0, 0.01));
//      spDP = new JSpinner(new SpinnerNumberModel(+0.02, 0.0, 10.0, 0
.01));
//      spDW = new JSpinner(new SpinnerNumberModel(+0.02, 0.0, 10.0, 0
.01));
//      spSimDT = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0,
0.1));
//      spDelay = new JSpinner(new SpinnerNumberModel(250, 20, 2000, 1
0));
//      cbRandomize = new JCheckBox("Randomize", true);
//      btnStart = new JButton("Start Simulation");
//      btnStop = new JButton("Stop Simulation");
//      btnReplay = new JButton("Replay from Log");
//
//      btnStart.addActionListener(e -> engine.setParams(
//          ((Number)spDT.getValue()).doubleValue(),
//          ((Number)spDH.getValue()).doubleValue(),
//          ((Number)spDP.getValue()).doubleValue(),
//          ((Number)spDW.getValue()).doubleValue(),
//          ((Number)spSimDT.getValue()).doubleValue(),
//          ((Number)spDelay.getValue()).longValue(),
//          cbRandomize.isSelected()
//      ));
//      btnStart.addActionListener(e -> engine.start());
//      btnStop.addActionListener(e -> engine.stop());
//      btnReplay.addActionListener(e -> new Thread(() -> replay.repla
y(Logger.getFile(), 60), "Replay").start());
//
//      sim.add(new JLabel("ΔT per tick (°C)")); sim.add(spDT);
//      sim.add(new JLabel("ΔH per tick (%)")); sim.add(spDH);
//      sim.add(new JLabel("ΔP per tick (mm/h)")); sim.add(spDP);
//      sim.add(new JLabel("ΔWind per tick (m/s)")); sim.add(spDW);
//      sim.add(new JLabel("dt (s)")); sim.add(spSimDT);
//      sim.add(new JLabel("Delay (ms)")); sim.add(spDelay);
//      sim.add(new JLabel(" ")); sim.add(cbRandomize);
//      sim.add(btnStart); sim.add(btnStop);
//      sim.add(new JLabel(" ")); sim.add(btnReplay);
//
//      left.add(Box.createVerticalStrut(6));
//      left.add(titled("Simulation", sim));

```

```

        JPanel sim = new JPanel(new GridLayout(0,2,6,6));
        spDT = new JSpinner(new SpinnerNumberModel(-0.02, -
5.0, 5.0, 0.01));
        spDH = new JSpinner(new SpinnerNumberModel(+0.10, -
5.0, 5.0, 0.01));
        spDP = new JSpinner(new SpinnerNumberModel(+0.02, 0.0, 10.0, 0.
01));
        spDW = new JSpinner(new SpinnerNumberModel(+0.02, 0.0, 10.0, 0.
01));
        spSimDT = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0
.1));
        spDelay = new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10
));

        cbRandomize = new JCheckBox("Randomize", true);
        btnStart = new JButton("Start Simulation");
        btnStop = new JButton("Stop Simulation");
        btnReplay= new JButton("Replay from Log");
        btnStart.addActionListener(e -> engine.setParams(
            ((Number)spDT.getValue()).doubleValue(),
            ((Number)spDH.getValue()).doubleValue(),
            ((Number)spDP.getValue()).doubleValue(),
            ((Number)spDW.getValue()).doubleValue(),
            ((Number)spSimDT.getValue()).doubleValue(),
            ((Number)spDelay.getValue()).longValue(),
            cbRandomize.isSelected()
        ));
        btnStart.addActionListener(e -> engine.start());
        btnStop.addActionListener(e -> engine.stop());
        btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
        sim.add(new JLabel("ΔT per tick (°C)")); sim.add(spDT);
        sim.add(new JLabel("ΔH per tick (%"));    sim.add(spDH);
        sim.add(new JLabel("ΔP per tick (mm/h)"));sim.add(spDP);
        sim.add(new JLabel("ΔWind per tick (m/s)")); sim.add(spDW);
        sim.add(new JLabel("dt (s)"));            sim.add(spSimDT);
        sim.add(new JLabel("Delay (ms)"));        sim.add(spDelay);
        sim.add(new JLabel(" "));                sim.add(cbRandomize);
        sim.add(btnStart);                        sim.add(btnStop);
        sim.add(new JLabel(" "));                sim.add(btnReplay);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Simulation", sim));
        getContentPane().add(left, BorderLayout.WEST);
        /* CENTER: Sensors + Alerts */
        JPanel center = new JPanel(new BorderLayout(6,6));

```

```

        tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
            @Override public boolean isCellEditable(int row, int column
) { return false; }
        };
        tblSensors = new JTable(tblModel);
        JScrollPane spTbl = new JScrollPane(tblSensors);
        center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
        JTextArea txtAlerts = new JTextArea(8, 60);
        txtAlerts.setEditable(false);
        center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
        getContentPane().add(center, BorderLayout.CENTER);
        /* RIGHT: Log tail */
        txtLog = new JTextArea();
        txtLog.setEditable(false);
        JScrollPane spLog = new JScrollPane(txtLog);
        getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);
        javax.swing.Timer alertsTimer = new javax.swing.Timer(1000, e -
> {
            StringBuilder sb = new StringBuilder();
            for (RAAlertSystem.Alert a : alerts.recent(18)) {
                sb.append(a).append("\n");
            }
            txtAlerts.setText(sb.toString());
            txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
        });
        alertsTimer.start();
    }
    private void refreshUi() {
        lblRoadT.setText(RStr.fmt2(road.getCurrentTemperature()));
        lblFreeze.setText(RStr.fmt2(road.getFreezingPoint()));
        lblStop.setText(RStr.fmt2(road.getStopTemp()));
        lblDisp.setText(road.isDispenserActive() ? "ON" : "OFF");
        lblDisp.setForeground(road.isDispenserActive() ? new Color(0,12
0,0) : Color.RED);
        //      lblAmbT.setText(RStr.fmt2(road.getAmbientTemperature()));
        //      lblHum.setText(RStr.fmt2(road.getHumidity()));
        //      lblPrec.setText(RStr.fmt2(road.getPrecipitationRate()));
        //      lblSnow.setText(road.isSnowing() ? "YES" : "NO");
        //      lblFr.setText(RStr.fmt2(road.getFriction()));
        //      lblWind.setText(RStr.fmt2(road.getWindSpeed()));
    }

```

```

//      lblSalt.setText(RStr.fmt2(road.getSaltKg()));
//      lblCost.setText(RStr.fmt2(road.getSaltCost()));

    lblAmbT.setText(RStr.fmt2(road.getAmbientTemperature()));
    lblHum.setText(RStr.fmt2(road.getHumidity()));
    lblPrec.setText(RStr.fmt2(road.getPrecipitationRate()));
    lblSnow.setText(road.isSnowing() ? "YES" : "NO");
    lblFr.setText(RStr.fmt2(road.getFriction()));
    lblWind.setText(RStr.fmt2(road.getWindSpeed()));
    lblSalt.setText(RStr.fmt2(road.getSaltKg()));
    lblCost.setText(RStr.fmt2(road.getSaltCost()));
    tblModel.setRowCount(0);
    tblModel.addRow(new Object[]{
        road.tempSensor().getName(),
        RStr.fmt2(road.tempSensor().getValue()),
        road.tempSensor().isOk(), road.tempSensor().getStatus()
    });
    tblModel.addRow(new Object[]{
        road.humiditySensor().getName(),
        RStr.fmt2(road.humiditySensor().getValue()),
        road.humiditySensor().isOk(), road.humiditySensor().get
Status()
    });
    String precipTag = road.precipSensor().isSnowing() ? " (snow)"
:
        (road.getPrecipitationRate() > 0 ? " (rain)" : " (none)
");
    tblModel.addRow(new Object[]{
        road.precipSensor().getName(),
        RStr.fmt2(road.precipSensor().getValue()) + precipTag,
        road.precipSensor().isOk(), road.precipSensor().getStat
us()
    });
    tblModel.addRow(new Object[]{
        road.frictionSensor().getName(),
        RStr.fmt2(road.frictionSensor().getValue()),
        road.frictionSensor().isOk(), road.frictionSensor().get
Status()
    });
    List<String> lines = logger.recent(220);
    StringBuilder sb = new StringBuilder();
    for (String s : lines) sb.append(s).append("\n");
    txtLog.setText(sb.toString());
    txtLog.setCaretPosition(txtLog.getDocument().getLength());

```

```

    }
}
/* ===== CLI Harness ===== */
class RoadCLIHarness {
    private final SmartRoadSafety road;
    RoadCLIHarness(SmartRoadSafety road) { this.road = road; }
    void runAll() {
        System.out.println("==== CLI Demo (VT) =====");
        vt();
        System.out.println("==== CLI Demo (FT) =====");
        ft();
        System.out.println("==== CLI Demo (Z3 mimic) =====");
        z3();
        System.out.println("==== CLI Demo (UVT) =====");
        uvt();
    }
    /** VT: typical boundaries to hit < freeze, between, and > stop. */
    void vt() {
        // initial 3.0 (example in main), freeze=0, stop=5
        road.updateRoadTemperature(-4.0); // 3.0 -> -
1.0 (< freeze) => ON
        road.updateRoadTemperature(+3.0); // -
1.0 -> 2.0 (between) => keep
        road.updateRoadTemperature(+4.0); // 2.0 -> 6.0 (> stop) => OFF
        road.updateRoadTemperature(-7.0); // 6.0 -> -
1.0 (< freeze) => ON
    }
    /** FT: extreme numeric ranges. */
    void ft() {
        road.updateRoadTemperature(0); // clamp by model semantics (no
clamp here, but we keep for edge)
        road.updateRoadTemperature(0);
        road.updateRoadTemperature(0);
        road.updateRoadTemperature(3.6603310227959246E298);
    }
    /** Z3-like sequence, same logical coverage as VT. */
    void z3() {
        road.updateRoadTemperature(-4.0);
        road.updateRoadTemperature(+3.0);
        road.updateRoadTemperature(+4.0);
        road.updateRoadTemperature(-7.0);
    }
    /** UVT: user validation test, mirror of VT. */
    void uvt() {

```

```

        road.updateRoadTemperature(-4.0);
        road.updateRoadTemperature(+3.0);
        road.updateRoadTemperature(+4.0);
        road.updateRoadTemperature(-7.0);
    }
}
/* ===== Main ===== */
public class c127_SmartRoadSafetySystemEx {
    public static void main(String[] args) {
        // Switch GUI/CLI by flag or headless env
        boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;
        // Logger & alerts
        RDataLogger logger = new RDataLogger("road_log.txt", 600);
        RAlertSystem alertSystem = new RAlertSystem(logger);
        // Core
        SmartRoadSafety road = new SmartRoadSafety(0.0, 5.0, 3.0, logger, alertSystem);
        // Engines
        WeatherSimulationEngine engine = new WeatherSimulationEngine(road, logger);
        RoadReplayEngine replay = new RoadReplayEngine(road, logger);
        // ===== CLI path (comment-to-toggle 4 groups) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            RoadCLIHarness harness = new RoadCLIHarness(road);
            // Run four groups sequentially; comment out 3 lines to run
            // single group
            //      harness.vt();    // ← keep this line to run VT
            harness.ft();    // ← comment out to skip FT
            //      harness.z3();    // ← comment out to skip Z3
            //      harness.uvt();    // ← comment out to skip UVT
            // Optional: a short weather simulation to exercise additional branches
            for (int i = 0; i < 20; i++) {
                road.applyWeather(-
0.05, +0.2, +0.05, (i % 5 == 0), +0.03);
                road.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {}
            }
            logger.log("CLI demo finished.");
            System.out.println("CLI demo finished. Log written to: " +
logger.getFile().getAbsolutePath());

```

```

        return;
    }
    // ===== GUI path =====
    logger.log("Running in GUI mode.");
    SwingUtilities.invokeLater(() -> {
        RoadGUI gui = new RoadGUI(road, logger, alertSystem, engine
, replay);
        gui.setVisible(true);
    });
}
private static boolean headless() {
    try { return GraphicsEnvironment.isHeadless(); }
    catch (Throwable t) { return true; }
}
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c127_SmartRoadSafetySystemEx.java	38.5 %	40	64	104
> RAlertSystem	0.0 %	0	2	2
> RoadReplayEngine	0.0 %	0	8	8
> WeatherSimulationEngine	0.0 %	0	10	10
> RDataLogger	25.0 %	2	6	8
> FrictionSensor	50.0 %	6	6	12
> HumiditySensor	50.0 %	3	3	6
> RoadTemperatureSensor	50.0 %	3	3	6
> c127_SmartRoadSafetySystemEx	62.5 %	5	3	8
> SmartRoadSafety	66.7 %	12	6	18
> SaltDispenserController	83.3 %	5	1	6
▼ PrecipitationSensor	100.0 %	4	0	4
▲ update(RoadContext)	100.0 %	4	0	4
▲ PrecipitationSensor()		0	0	0
> RoadCLIHarness		0	0	0
> RoadContext		0	0	0
> RSensor		0	0	0

3. Vibration testing instruction coverage: 93.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c127_SmartRoadSafetySystemEx.java	32.4 %	1,053	2,193	3,246
> RoadContext	6.1 %	3	46	49
> RoadReplayEngine	6.5 %	9	129	138
> WeatherSimulationEngine	14.8 %	41	236	277
> RoadCLIHarness	24.2 %	23	72	95
> RStr	54.5 %	12	10	22
> RAlertSystem	55.9 %	33	26	59
> SaltDispenserController	56.2 %	63	49	112
> RDataLogger	57.9 %	70	51	121
> RSensor	69.0 %	20	9	29
> c127_SmartRoadSafetySystemEx	70.1 %	82	35	117
> SmartRoadSafety	73.6 %	418	150	568
> FrictionSensor	88.2 %	82	11	93
▼ PrecipitationSensor	93.5 %	43	3	46
▲ PrecipitationSensor()	100.0 %	4	0	4
▲ update(RoadContext)	100.0 %	39	0	39
> HumiditySensor	95.5 %	42	2	44

4. Fuzz testing branch coverage: 75%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c127_SmartRoadSafetySystemEx.java		35.6 %	37	67	104
> RAlertSystem		0.0 %	0	2	2
> RoadReplayEngine		0.0 %	0	8	8
> WeatherSimulationEngine		0.0 %	0	10	10
> RDataLogger		25.0 %	2	6	8
> FrictionSensor		50.0 %	6	6	12
> HumiditySensor		50.0 %	3	3	6
> SmartRoadSafety		50.0 %	9	9	18
> c127_SmartRoadSafetySystemEx		62.5 %	5	3	8
> SaltDispenserController		66.7 %	4	2	6
▼ PrecipitationSensor		75.0 %	3	1	4
▲ update(RoadContext)		75.0 %	3	1	4
▲ PrecipitationSensor()			0	0	0
> RoadTemperatureSensor		83.3 %	5	1	6
> RoadCLIHarness			0	0	0
> RoadContext			0	0	0
> RSensor			0	0	0

5. Fuzz testing instruction coverage: 91.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c127_SmartRoadSafetySystemEx.java		31.6 %	1,025	2,221	3,246
> RoadContext		6.1 %	3	46	49
> RoadReplayEngine		6.5 %	9	129	138
> WeatherSimulationEngine		14.8 %	41	236	277
> RoadCLIHarness		24.2 %	23	72	95
> SaltDispenserController		44.6 %	50	62	112
> RStr		54.5 %	12	10	22
> RAlertSystem		55.9 %	33	26	59
> RDataLogger		57.9 %	70	51	121
> RSensor		69.0 %	20	9	29
> c127_SmartRoadSafetySystemEx		70.1 %	82	35	117
> SmartRoadSafety		70.8 %	402	166	568
> FrictionSensor		88.2 %	82	11	93
▼ PrecipitationSensor		91.3 %	42	4	46
▲ update(RoadContext)		97.4 %	38	1	39
▲ PrecipitationSensor()		100.0 %	4	0	4

6. Z3 branch coverage: 100%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c127_SmartRoadSafetySystemEx.java		38.5 %	40	64	104
> RAlertSystem		0.0 %	0	2	2
> RoadReplayEngine		0.0 %	0	8	8
> WeatherSimulationEngine		0.0 %	0	10	10
> RDataLogger		25.0 %	2	6	8
> FrictionSensor		50.0 %	6	6	12
> HumiditySensor		50.0 %	3	3	6
> RoadTemperatureSensor		50.0 %	3	3	6
> c127_SmartRoadSafetySystemEx		62.5 %	5	3	8
> SmartRoadSafety		66.7 %	12	6	18
> SaltDispenserController		83.3 %	5	1	6
▼ PrecipitationSensor		100.0 %	4	0	4
▲ update(RoadContext)		100.0 %	4	0	4
▲ PrecipitationSensor()			0	0	0
> RoadCLIHarness			0	0	0
> RoadContext			0	0	0
> RSensor			0	0	0

7. Z3 instruction coverage: 93.5%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c127_SmartRoadSafetySystemEx.java		32.4 %	1,053	2,193	3,246
> RoadContext		6.1 %	3	46	49
> RoadReplayEngine		6.5 %	9	129	138
> WeatherSimulationEngine		14.8 %	41	236	277
> RoadCLIHarness		24.2 %	23	72	95
> RStr		54.5 %	12	10	22
> RAlertSystem		55.9 %	33	26	59
> SaltDispenserController		56.2 %	63	49	112
> RDataLogger		57.9 %	70	51	121
> RSensor		69.0 %	20	9	29
> c127_SmartRoadSafetySystemEx		70.1 %	82	35	117
> SmartRoadSafety		73.6 %	418	150	568
> FrictionSensor		88.2 %	82	11	93
▼ PrecipitationSensor		93.5 %	43	3	46
▲ PrecipitationSensor()		100.0 %	4	0	4
▲ update(RoadContext)		100.0 %	39	0	39
> HumiditySensor		95.5 %	42	2	44

8. Unimproved V-method branch coverage: 100%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c127_SmartRoadSafetySystemEx.java		38.5 %	40	64	104
> RAlertSystem		0.0 %	0	2	2
> RoadReplayEngine		0.0 %	0	8	8
> WeatherSimulationEngine		0.0 %	0	10	10
> RDataLogger		25.0 %	2	6	8
> FrictionSensor		50.0 %	6	6	12
> HumiditySensor		50.0 %	3	3	6
> RoadTemperatureSensor		50.0 %	3	3	6
> c127_SmartRoadSafetySystemEx		62.5 %	5	3	8
> SmartRoadSafety		66.7 %	12	6	18
> SaltDispenserController		83.3 %	5	1	6
▼ PrecipitationSensor		100.0 %	4	0	4
▲ update(RoadContext)		100.0 %	4	0	4
▲ PrecipitationSensor()			0	0	0
> RoadCLIHarness			0	0	0
> RoadContext			0	0	0
> RSensor			0	0	0

9. Unimproved V-method instruction coverage: 93.5%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c127_SmartRoadSafetySystemEx.java		32.4 %	1,053	2,193	3,246
> RoadContext		6.1 %	3	46	49
> RoadReplayEngine		6.5 %	9	129	138
> WeatherSimulationEngine		14.8 %	41	236	277
> RoadCLIHarness		24.2 %	23	72	95
> RStr		54.5 %	12	10	22
> RAlertSystem		55.9 %	33	26	59
> SaltDispenserController		56.2 %	63	49	112
> RDataLogger		57.9 %	70	51	121
> RSensor		69.0 %	20	9	29
> c127_SmartRoadSafetySystemEx		70.1 %	82	35	117
> SmartRoadSafety		73.6 %	418	150	568
> FrictionSensor		88.2 %	82	11	93
> PrecipitationSensor		93.5 %	43	3	46
> HumiditySensor		95.5 %	42	2	44
> RoadTemperatureSensor		96.6 %	56	2	58